Generating random variables.

Covered by chapter 3.1-3.4 in the textbook.

A key part of statistical simulation which we shall consider now is how to simulate data from distributions with a specified pdf/pmf $f(x)$.

Generating uniform pseudo random numbers

The basis for simulation of random variables is to have a good uniform random number generator available. For true random number generation a generator based on some physical phenomena like thermodynamic noise, radioactivity, atmospheric noise, etc can be used. However, for our applications in simulation and modelling such generators are too slow. We thus have to resort to *pseudo random number generators* (PRNGs). PRNGs are generated by mathematical formulas to mimic the properties of true random numbers as good as possible.

PRNGs are deterministic in the sense that from a given starting point ("seed") the same sequence will always be generated. In R, first repeat `runif(10)` several times. Then repeat: `set.seed(98925)` followed by `runif(10)`

There exist many PRNGs, and most of them generate an apparently random sequence of integers (which by dividing by the maximum number gives a sequence of standard uniform numbers). One of the best in terms of both good randomness properties and speed is an algorithm called Mersenne Twister which is the default PRNG in R (and many other programming languages).

For more background on PRNG see the help pages for random number generation in R (type e.g. `?RNG` in R) and the Wikipedia pages `Pseudorandom_number_generator`. The brief introduction `www.random.org/randomness/` is also instructive.

The `runif` function in R which generate PRNGs from the (standard) uniform distribution will be our basic random number generator.

The inverse transform method, continuous case

Challenge: How to simulate data from a continuous distribution with pdf $f(x)$?

If it is easy to invert the cdf $F(x)$ it is easy to simulate from the distribution due to the following result (often called the probability integral transformation):

Theorem: If X is a cont. random variable with cdf $F(x)$ then $U = F(X) \sim \text{Uniform}[0, 1]$.

□

Hints that we may use $X = F^{-1}(U)$ as a way to generate $X \sim F$:

Proof:

$$\begin{aligned} P(X \leq x) &= P(F^{-1}(U) \leq x) = P(U \leq F(x)) \\ &= F(x) \end{aligned}$$

□

Algorithm:

1. Generate $u \sim \text{Uniform}[0, 1]$
2. Let $x = F^{-1}(u)$ □

Example: Exponential distribution with pdf

$$f(x) = \lambda e^{-\lambda x}, \quad x > 0.$$

CDF:

$$F(x) = \int_0^x f(t)dt = 1 - \exp(-\lambda x)$$

Now solve the equation $F(X) = U$ for X to get the inverse CDF:

$$1 - \exp(-\lambda X) = U \Rightarrow X = -\lambda^{-1} \log(1 - U)$$

(notice that also $1 - U \sim \text{Uniform}[0, 1]$, hence $\lambda^{-1} \log(U)$ also gives correct distribution)

See R-code: `inverse_transform_examples.R`

Example: General uniform distribution
Suppose we are interested in random draws from Uniform$[a, b]$, but only have Uniform$[0, 1]$ draws available:

$$F(x) = \int_a^x \frac{1}{b-a} dt = \frac{x-a}{b-a}, \; a \leq x \leq b.$$

Solve:

$$\frac{x-a}{b-a} = U \Rightarrow X = a + (b-a)U$$

In principle, the "known" distributions in R may be simulated using the quantile (q) functions $(= F^{-1})$ e.g.

```
X <- qnorm(p=runif(n),mean=mu,sd=sigma)
```

However, this practice is not recommended in a computational efficiency perspective.

The inverse transform method, discrete case

Challenge: How to simulate data from a discrete distribution with pmf $f(x)$?

The inverse transform method also works for discrete distributions. Let $x_1 < x_2 < x_3 < \ldots$ denote the possible values of the discrete r.v. X. Then:

Algorithm:

1. Generate $u \sim \text{Uniform}[0, 1]$
2. Let $X = x_i$ where $F(x_{i-1}) < u \leq F(x_i)$ $\square$

However, do not implement this procedure on your own. Rather use the `sample`-function, e.g.:

```
x <- 0:3 # possible outcomes
n <- 1000 # number of draws
probs <- c(1,3,3,1)/8 # probability of outcome
X <- sample(x=x,size=n,replace=TRUE,prob=probs)
```

Example: Let X="number of heads in three throws of a coin". Then we have seen earlier:

x	0	1	2	3
$f(x)$	1/8	3/8	3/8	1/8

Use the inverse transform method to generate data from this pmf.

```
x <- 0:3 # possible outcomes
n <- 1000 # number of draws
probs <- c(1,3,3,1)/8 # probability of outcome
X <- sample(x=x,size=n,replace=TRUE,prob=probs)
```

See further examples in chap 3.2 in the book.

The acceptance-rejection method

Challenge: How to simulate from a pdf/pmf $f(x)$?

The inverse transform method is very efficient when the inverse transform is easy to evaluate. However, for many distribution the inversion is cumbersome. In some such cases the acceptance-rejection method can be an alternative

Acceptance-rejection sampling applies when X has a pdf/pmf $f(x)$ and Y has a pdf/pmf $g(y)$ and there exist a constant c such that

$$\frac{f(t)}{g(t)} \leq c$$

for all t where $f(t) > 0$. For best performance g should be chosen such that it is easy to simulate from and such that c is small.

Algorithm:

1. Generate y from g.
2. Generate $u \sim \text{Uniform}[0, 1]$
3. If $u < f(y)/(cg(y))$ accept y and let $x = y$. Otherwise, reject y and repeat from step 1.

Why does this work?

Notice first that by step 3 the chance of accepting a proposal y is:

$$P(\text{accept}|Y = y) = P\left(U < \frac{f(y)}{cg(y)}\right) = \frac{f(y)}{cg(y)}$$

Then overall (in the discrete case):

$$\begin{aligned} P(\text{accept}) &= \sum_y P(\text{accept}|Y = y)P(Y = y) \\ &= \sum_y \frac{f(y)}{cg(y)} g(y) = \frac{1}{c} \end{aligned}$$

(the argument is the same for the continuous case just with integration).

Thus the number of iterations of the algorithm until acceptance has a geometric distribution with expectation $1/P(\text{accept}) = c$.

Finally, an accepted value has the correct pmf because (similar argument for the pdf in the continuous case):

$$\begin{aligned} P(Y = x|\text{accept}) &= \frac{P(\text{accept}|Y = x)P(Y = x)}{P(\text{accept})} \\ &= \frac{[f(x)/(cg(x))]g(x)}{1/c} = f(x) \end{aligned}$$

Example: Use acceptance-rejection to sample data from the pdf in problem 5 in exercise set 1.

$$f(x) = \begin{cases} 0.75(1-x^2) & \text{for } -1 \leq x \leq 1, \\ 0 & \text{otherwise,} \end{cases}$$

We will use a Uniform$[-1, 1]$-proposal with $g(x) = 0.5$. Now $f(x)$ is maximized at $x = 0$, with $f(0) = 0.75$, hence

$$\frac{f(x)}{g(x)} = 2f(x) \leq 2 \cdot 0.75 = 1.5$$

See R-code: `AR-example.R`

Transformation methods

There are several relations between distribution which can be used for simulating random variables. See page 58 in the book for some examples.

It can for instance be shown that if $U \sim \text{Gamma}(\alpha_1, \beta)$ and $V \sim \text{Gamma}(\alpha_2, \beta)$ then $U/(U+V)$ has a $\text{Beta}(\alpha_1, \alpha_2)$ distribution. Thus if we have a way of generating data from the gamma distribution this relation can be used to generate data from the beta distribution. See details in chap 3.4 in the book.